

```

# Step 1 Loading the data
FUNCTION LOAD_DATA()
    RETURN hardcoded list of market price records
END FUNCTION

```

```

# Step 2 Standardizing the data
FUNCTION STANDARDIZE_RECORD(record)
    IF market = "wakulima"
        SET market = "Wakulima"
    IF market = "Nakuru top market"
        SET market = "Nakuru Top Market"
    IF unit = "KG" OR unit = "Kgs" OR unit = "kilogram"
        SET unit = "kg"
    IF unit = "90 kg bag"
        SET unit = "90kg bag"
    RETURN record
END FUNCTION

```

```

# Step 3 Validating Records
FUNCTION VALIDATE_RECORD(record)
    IF record ID is missing
        RETURN False, "Missing Record ID"
    ELSE IF market is missing
        RETURN False, "Missing Market"
    ELSE IF commodity is missing
        RETURN False, "Missing Commodity"
    ELSE IF unit is not "kg" or "90kg bag"
        RETURN False, "Unsupported Unit"
    ELSE IF price is not numeric OR price <= 0
        RETURN False, "Invalid Price"
    ELSE IF quantity is not numeric OR quantity < 0
        RETURN False, "Invalid Quantity"
    ELSE
        RETURN True, None
    END IF
END FUNCTION

```

```

# Step 4 Derived Values
FUNCTION CALCULATE_DERIVED_FIELDS(record)
    IF unit = "kg"
        SET price_per_kg = price
        SET quantity_kg = quantity
    ELSE IF unit = "90kg bag"
        SET price_per_kg = price / 90
        SET quantity_kg = quantity * 90
    END IF
    STORE price_per_kg and quantity_kg in record
    RETURN record
END FUNCTION

```

```

# Step 5 Main Program flow
FUNCTION PREPARE_RECORDS()
    raw_records ← LOAD_DATA()
    CREATE empty list for valid records
    CREATE empty list for invalid records

```

```

FOR each record in raw_records
    record ← STANDARDIZE_RECORD(record)
    is_valid, reason ← VALIDATE_RECORD(record)

    IF is_valid
        record ← CALCULATE_DERIVED_FIELDS(record)
        ADD record to valid records
    ELSE
        STORE reason in record as rejection_reason
        ADD record to invalid records
    END IF
END FOR

RETURN valid records, invalid records
END FUNCTION

#Step 6 View Available Commodities
FUNCTION view_commodities(valid_records):
    IF valid_records is empty THEN
        PRINT "No valid records available."
        RETURN
    END IF

    CREATE empty set commodities

    FOR EACH record IN valid_records
        ADD record.commodity TO commodities
    END FOR

    SET sorted_commodities = SORT(commodities) ALPHABETICALLY

    PRINT "Available commodities:"

    FOR EACH commodity IN sorted_commodities
        PRINT " - " + commodity
    END FOR
END FUNCTION

#Step 7 View Prices for a Commodity
FUNCTION view_prices_for_commodity(valid_records):
    IF valid_records is empty:
        PRINT "No valid records available."
        RETURN

    CALL view_commodities(valid_records)

    commodity = INPUT "Enter commodity: "
    commodity = REMOVE leading and trailing spaces from commodity

    selected_records = empty list

    FOR each record in valid_records:

        IF record["commodity"] matches commodity:
            ADD record to selected_records

    IF selected_records is empty:
        PRINT "Commodity not found."
        RETURN

```

```

SORT selected_records by price_per_kg from lowest to highest

PRINT "Prices for selected commodity:"

FOR each record in selected_records:

    IF record["unit"] is "kg":
        price_per_kg = record["price_kes"]

    ELSE IF record["unit"] is "90kg bag":
        price_per_kg = record["price_kes"] / 90

    DISPLAY market,county, unit, price and price_per_kg

END FUNCTION

## Step 8. Analyse a commodity
FUNCTION analyse_commodity(valid_records):

    IF valid_records is empty:
        PRINT "No valid records available."
        RETURN

    CALL view_commodities(valid_records)

    commodity = INPUT "Enter commodity to analyse: "
    commodity = REMOVE leading and trailing spaces from commodity

    matches = empty list

    FOR each record in valid_records:
        IF record["commodity"] (case-insensitive) matches commodity:
            ADD record to matches

    IF matches is empty:
        PRINT 'No valid records found for "[commodity]".'
        RETURN

    # Calculate statistics
    min_price = matches[0]["price_per_kg"]
    max_price = matches[0]["price_per_kg"]
    cheapest_market = matches[0]["market"]
    priciest_market = matches[0]["market"]
    total_price = 0

    FOR each record in matches:
        IF record["price_per_kg"] < min_price:
            min_price = record["price_per_kg"]
            cheapest_market = record["market"]

        IF record["price_per_kg"] > max_price:
            max_price = record["price_per_kg"]
            priciest_market = record["market"]

        total_price = total_price + record["price_per_kg"]

    avg_price = total_price / LENGTH(matches)

```

```

above_avg_count = 0
below_avg_count = 0

FOR each record in matches:
    IF record["price_per_kg"] > avg_price:
        above_avg_count = above_avg_count + 1

    IF record["price_per_kg"] < avg_price:
        below_avg_count = below_avg_count + 1

# Display analysis
PRINT "--- ANALYSIS: [commodity] ---"
PRINT "Minimum price : KES " + ROUND(min_price, 2) + "/kg"
PRINT "Maximum price : KES " + ROUND(max_price, 2) + "/kg"
PRINT "Average price : KES " + ROUND(avg_price, 2) + "/kg"
PRINT "Cheapest market      : " + cheapest_market
PRINT "Most expensive market : " + priciest_market
PRINT "Records above average : " + above_avg_count
PRINT "Records below average : " + below_avg_count

# Display sorted records
SORT matches by price_per_kg from lowest to highest

PRINT "--- [commodity] PRICES (lowest to highest) ---"
DISPLAY TABLE with columns: Market, County, Unit, Price(KES), KES/kg

FOR each record in matches:
    PRINT market, county, unit, price_kes, price_per_kg

END FUNCTION

## Step 9. Compare two markets
FUNCTION COMPARE_MARKETS():
    DISPLAY "Available commodities"

    GET list of unique commodities from valid records
    SORT commodities alphabetically

    DISPLAY commodities

    INPUT selected_commodity

    IF selected_commodity does not exist THEN
        DISPLAY "Invalid commodity"
        RETURN
    END IF

    GET all markets available for the selected commodity
    SORT markets alphabetically

    DISPLAY available_markets

    INPUT first_market

    IF first_market does not exist THEN
        DISPLAY "Invalid first market"
        RETURN
    END IF

```

```

INPUT second market

IF second market does not exist THEN
    DISPLAY "Invalid second market"
    RETURN
END IF

IF first market = second market THEN
    DISPLAY "Please select two different markets"
    RETURN
END IF

FIND record for selected commodity in first market
FIND record for selected commodity in second market

IF first market record does not exist THEN
    DISPLAY "No valid record found for first market"
    RETURN
END IF

IF second market record does not exist THEN
    DISPLAY "No valid record found for second market"
    RETURN
END IF

IF first market unit = "kg" THEN
    first price per kg = first market price

ELSE IF first market unit = "90kg bag" THEN
    first price per kg = first market price / 90
END IF

IF second market unit = "kg" THEN
    second price per kg = second market price

ELSE IF second market unit = "90kg bag" THEN
    second price per kg = second market price / 90
END IF

CALCULATE price difference =
    ABS(first price per kg - second price per kg)

IF first price per kg < second price per kg THEN
    cheaper market = first market

ELSE IF second price per kg < first price per kg THEN
    cheaper market = second market

ELSE
    cheaper market = "Both markets have the same price"
END IF

DISPLAY selected commodity
DISPLAY first market and price per kg

```

```
    DISPLAY second market and price per kg
    DISPLAY price difference
    DISPLAY cheaper market
```

```
END FUNCTION
```

```
##Step 10: DATA SUMMARY
```

```
FUNCTION DATA_SUMMARY():
```

```
    Count all records
```

```
    Count valid records
```

```
    Count invalid records
```

```
    Set total quantity to zero
```

```
    Set total quantity in kg to zero
```

```
    Set kg record count to zero
```

```
    Set 90kg-bag record count to zero
```

```
    FOR every valid record
```

```
        Add quantity to total quantity
```

```
        Add converted quantity in kg to total quantity in kg
```

```
        IF unit is kg
```

```
            Add 1 to kg record count
```

```
        ELSE IF unit is 90kg bag
```

```
            Add 1 to 90kg-bag record count
```

```
    END FOR
```

```
    Display:
```

```
        Total records
```

```
        Valid records
```

```
        Invalid records
```

```
        Total quantity
```

```
        Total quantity in kg
```

```
        Number of kg records
```

```
        Number of 90kg-bag records
```

```
END FUNCTION
```

```
##Step 11: View Invalid Records
```

```
FUNCTION VIEW_INVALID_RECORDS(invalid_records):
```

```
    IF invalid_records is empty THEN
```

```
        PRINT "No invalid records found. All records passed  
validation."
```

```
        RETURN
```

```
    END IF
```

```
    PRINT "--- INVALID RECORDS ---"
```

```
    FOR each record in invalid_records
```

```

        PRINT record["record_id"], record["market"],
record["commodity"],
        record["unit"], record["rejection_reason"]
    END FOR

    PRINT "Total invalid records: " + COUNT(invalid_records)

END FUNCTION

##Step 12: Main Menu (menu-control loop)

FUNCTION DISPLAY_MENU():
    PRINT "1. View available commodities"
    PRINT "2. View prices for a commodity"
    PRINT "3. Analyse a commodity"
    PRINT "4. Compare two markets"
    PRINT "5. View invalid records"
    PRINT "6. View dataset summary"
    PRINT "7. Exit"
END FUNCTION

FUNCTION RUN_MENU():
    valid_records, invalid_records ← PREPARE_RECORDS()

    WHILE True
        CALL DISPLAY_MENU()
        choice ← INPUT "Enter selection: "
        choice ← REMOVE leading and trailing spaces from choice

        IF choice = "1" THEN
            CALL view_commodities(valid_records)
        ELSE IF choice = "2" THEN
            CALL view_prices_for_commodity(valid_records)
        ELSE IF choice = "3" THEN
            CALL analyse_commodity(valid_records)
        ELSE IF choice = "4" THEN
            CALL COMPARE_MARKETS(valid_records)
        ELSE IF choice = "5" THEN
            CALL VIEW_INVALID_RECORDS(invalid_records)
        ELSE IF choice = "6" THEN
            CALL DATA_SUMMARY(valid_records, invalid_records)
        ELSE IF choice = "7" THEN
            PRINT "Program closed."
            BREAK
        ELSE
            PRINT "Invalid selection. Please try again."
        END IF
    END WHILE
END FUNCTION

```